

Recap

To recap: $u(t)$, in this case, is the attack signal. The assumption on the attacker is that we consider the most powerful attacker, so he can independently alter every equation.

Thus, $u \in \mathbb{R}^{n_x+n_y}$, with $B = [I \ 0]$ and $D = [0 \ I]$.

We said that control inputs are ignored because the term is known to the defender, and can be subtracted from the measurements. This is the superimposition principle for linear systems.

If an attacker takes control of one actuator in a real attack, however, this actuator may have impact on the first and last question, but the impact is given by the laws of physics. Here we study the case of the most powerful attack.

Yesterday we introduced the monitor, and we defined the consistency conditions. K was defined as the attack set, telling us which of the variables are being added to the equations describing the model.

$$\begin{array}{c}
 \downarrow u_k \\
 \boxed{\begin{cases} E\dot{x} = Ax + Bu \\ y = Cx + Du \\ x_0 \text{ known} \end{cases}} \\
 \downarrow y(t) \\
 \boxed{\Lambda = \{E, A, C, y(t) \forall t\}} \begin{array}{l} \longrightarrow \Phi_1 \\ \longrightarrow \Phi_2 \end{array}
 \end{array}$$

■

We also need to fix minimal rules for the detector, to have something useful in practice:

1. No false positives in detection. $\Phi_1(\Lambda) = \text{True}$ only if $K \neq \{\}$ (there is actually an attack).
2. "Internal consistency" of monitor. $\Phi_1(\Lambda) = \text{True} \iff \Phi_2(\Lambda) \neq \{\}$.
3. "Guarantee on simplest explanation for attack identification". It is about identification. $\Phi_2 = \hat{K}$ only if there is no other set S with equal or less entries that can explain the data. In formulas: $\Phi_2 = \hat{K}$ only if $\nexists S \subseteq \{1, 2, \dots, n_x + n_y\}$

: $|S| \leq |\hat{K}|$, $y(t) = y(x(0), u_{K_{[0,t]}}(t))$. This final element is the real explanation of the data, but the operator does not know this information. $y(t)$ should also be equal to $y(x', u_S, t)$.

Note: We assume that operator does not know the initial state of the system $x(0)$.

For example: if the monitor says at a certain point there is an attack ($\Phi_1 = \text{True}$), and provides $\Phi_2 = \{2, 4, 7\}$, then I'm sure that (by third property), that it's not possible to have simpler attacks (u_S , $|S| \leq |\hat{K}|$) that explain the data.

Φ_2 is guaranteed to return the simplest possible explanation for the data. In some cases, there are more than one set that can explain the data. We might have, for example, $\hat{K}_1 = \{2, 3, 4\}$ and $\hat{K}_2 = \{5, 7, 9\}$. In this case, the authors of the paper decided that the monitor will return all the variables: $\Phi_2 = \{1, 2, \dots, n_x + n_y\}$.

Thus, when $\Phi_1 = \text{True}$ we can trust it (by first property), but when $\Phi_1 = \text{False}$ it's not necessarily true that there is no attack, we might have false negatives.

Also, $\Phi_2 = \hat{K}$ does not guarantee that $\hat{K} = K$.

Let's define undetectable attacks: u_K is undetectable $\iff$ no consistent monitor can detect it. For every consistent monitor, $\Phi_1(\Lambda = \{A, E, C, y(x_0, u_k, t)\}) = \text{False}$. Unidentifiable attack: u_K is unidentifiable $\iff$ no consistent monitor can identify it.

Undetectable attacks are also unidentifiable by second property of monitors.

An attack set K is undetectable $\iff$ attacks built on it (u_k) are undetectable.

Now, let's see the condition on undetectable attacks.

An attack u_K (with $K \neq \{\}$) is undetectable (by consistent monitors) $\iff y(t) = y(t, x(0), u_K) = y(t, x'(0), 0) \forall t$. So data can be explained both by having an attack, but also from a different initial state with no attack. Data generated by the attack is compatible (can be explained as) with the data produced by the system in a normal operative condition (0 attack) starting from another initial state $x'(0)$. (Here we assume operator and monitor does not know the initial state).

Proof

($\implies$)

If $y(t)$ can be explained both by $y(t, x(0), u_K)$ and $y(t, x'(0), 0)$ then u_K is undetectable.

By the third property, when getting $y(t)$ any consistent monitor is forced to output $\Phi_2 = \{\}$, because since $y(t) = y(t, x', 0)$, the simplest explanation for the

observed data is the trivial attack (no attack) $K = \{\}$. So, having $\Phi_2 = \{\}$, by the second property if $\Phi_2 = \{\}$ then Φ_1 is false, making the attack undetectable.

($\Leftarrow$)

Prove that if u_k is undetectable, then $y(t) = y(t, x(0), u_K) = t(t, x'(0), 0) \forall t$.

By contradiction, by proving that $y(t, x(0), u_K) \neq y(t, x'(0), 0)$ for some t , then u_k is detectable. Suppose we are able to prove this, then the first is also true.

The second is proved by construction, specifically by designing a monitor that in this case ($y \neq y(x', t, 0)$ for some t) is able to detect the attack.



✓ Theorem (on unidentifiable attacks)

A non-zero attack u_K is not identifiable $\iff$
 $y(t) = t(t, x(0), u_K) = y(t, x'(0), u_S)$ with $|S| \leq |K|$.

Any time we can find a simple explanation, it means we cannot correctly identify the attack.

📖 Proof

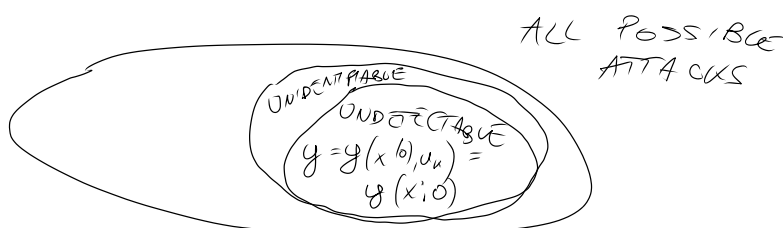
($\implies$)

Because of the third consistency property. We have two cases:

- If $|S| = |K|$, then $\Phi_2 = \{1, 2, \dots, n_x + n_y\}$.
- If $|S| < |K|$, then $\Phi_2 = S \neq K$.



To summarise:



The drawing pictures the hierarchy of attacks, where all possible attacks contain unidentifiable attacks, which also contains undetectable attacks.

In particular, an unidentifiable attack takes place when $\Phi_1 = \text{True}$ and $\Phi_2 = \{2, 3\}$ (for example), but the actual attack vector is more complex (for example: $K = \{2, 3, 5, 7\}$).

Let's now see an example of a typical monitor.

In general, we can have three types of monitors:

1. (Simplest) Static monitor. The outputs of the monitor $(\Phi_1(t), \Phi_2(t))$ only depend on the measurements at the same time, $y(t)$. (Bad data detector is an example of a static monitor).
2. Dynamical monitor. The outputs of the monitor depend also on past measurement (not just on current measurement). We see a classic example now.
3. Active monitor. Injects signals into the system to reveal attacks (watermarking).

In the figure above, static monitors will detect some attacks, while dynamical monitors will detect even more, but not undetectable attacks.

Now, let's just see the intuition of a typical passive dynamical monitor (we focus only on detection). A common strategy for detection uses observers.

$$\begin{array}{c}
 \downarrow u_x \\
 \boxed{
 \begin{array}{l}
 E\dot{x} = Ax + Bu \\
 y = Cx + Du \\
 x(0)
 \end{array}
 } \\
 \downarrow y(t) \forall t \\
 \boxed{
 \begin{array}{l}
 E\dot{w} = Aw + G(y - \hat{C}w) \\
 \left\{ \begin{array}{ll} \hat{y} = \hat{C}w & \text{estimated output} \\ r = y - \hat{y} & \text{residue} \end{array} \right. \\
 w(0) \text{ initial state of monitor}
 \end{array}
 } \longrightarrow r
 \end{array}$$

When we don't have the attack, the monitor is a copy of the system. We call the state of the monitor w . $\hat{y}$ as it's an estimate of the output. G multiplies the residue.

r is the output of the monitor, important for detection.

The monitor is the Luemberger observer.

Let's see how monitor works:

We start from a simple situation, in which we have no attack ($u_K = 0$) and we

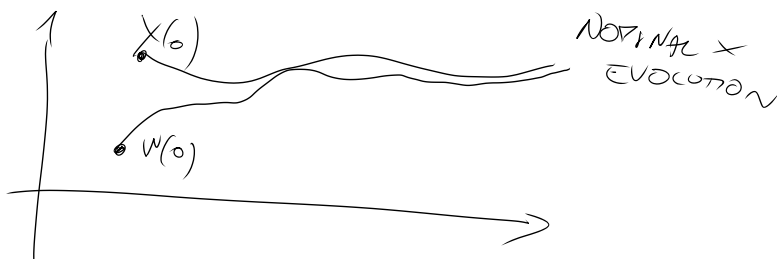
know the initial state ($x(0)$). We set $w(0) = x(0)$. What happens to the residual signal? It's zero, as $y = Cx$ and $\hat{y}$ is $Cw = Cx$, so $r = y - \hat{y} = Cx - Cx = 0$.

A more realistic normal condition sees:

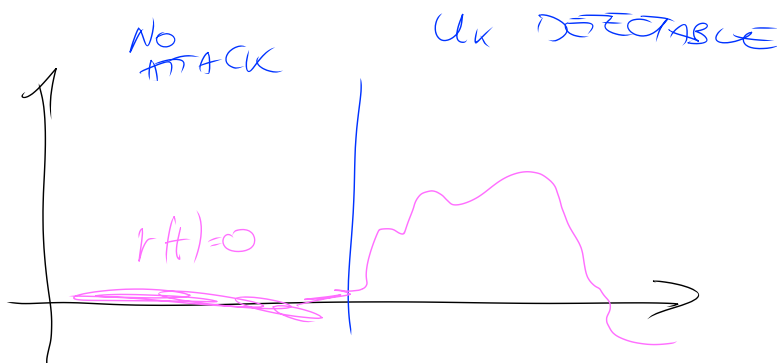
- No attack, $u_K(t) = 0 \forall t$.
- We don't know the initial state of the system, $x(0)$. So $w(0) \neq x(0)$.

$$E\dot{w} = Aw + G(y - \hat{y}) = Aw + G(Cx - Cw) = (A - GC)w + GCx$$

If G is chosen properly (all eigenvalues of $(A - GC)$ have negative real part), then $w(t) \rightarrow x(t)$.



Graph showing the function of nominal evolution of x , showing how, with time, the nominal evolution of w approaches the one of x .



In this kind of detection algorithms we have at least two components:

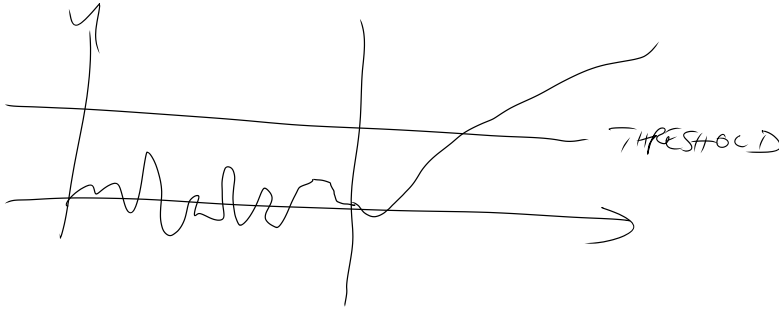
1. One used to generate the residual signal $r(t)$. In this case, we use an observer to do that.
2. Once found $r(t)$, we have to decide a detection rule. One of these is given by a threshold based detection rule. In general, however, r is a vector. In this case, the rule can either be on the single component of r , or on $\|r\|$, $\int_0^t \|r(z)\| dz$.

In reality, the second graph describes an ideal situation. In fact, we have:

- Model uncertainty: E, A, C are just an approximated model of the system.

- $w \neq 0$ (here w is not the state of the monitor, but the disturbance of the plant), $n \neq 0$ (sensor noise).

The ideal situation is given by:



The choice of the threshold is critical to avoid false positives.

So, for Φ_1 , we may have that it's true when, for example, $\|r(t)\| > 0.1$, while for Φ_2 it depends on the single observers on single variables.

Now, what are we going to see in the next lessons?

First, we'll take a look at False Data Injection Attacks against state estimation in power systems.

We have the grid from one side, which is a network of nodes (with links given by power lines, and nodes could be a city, etc.), and somewhere else the control center. The grid is described exactly as we've seen before, so
$$\begin{cases} E\dot{x} = Ax + Bu_c \\ y = Cx + Du_c \end{cases}.$$

The state, in this case, is given by voltage angles (phases). In the power grid we don't directly measure the voltage angles, rather we measure power. The vector of measurements z arrives in the control center. There is a Bad Data Detector, which has a scope of detecting errors in the communication link or in sensors. After the data is clean, it goes into a state estimation algorithm, which has a purpose of providing a state estimation $\hat{x}$. Then, the estimated state is used by multiple controllers. One might be used, for example, to give controls to a generator, like increase productions.

If we have an attack on the measurements, the state estimation will output a fake state estimation x_a , leading to possible wrong commands sent by controllers.